Lab 5: Vibe-coding B

- ✓ 提示词工程prompt engineering
- ✓ 在Vibe-coding使用版本控制
- ✓ 测试驱动的Vibe-coding

提示词工程

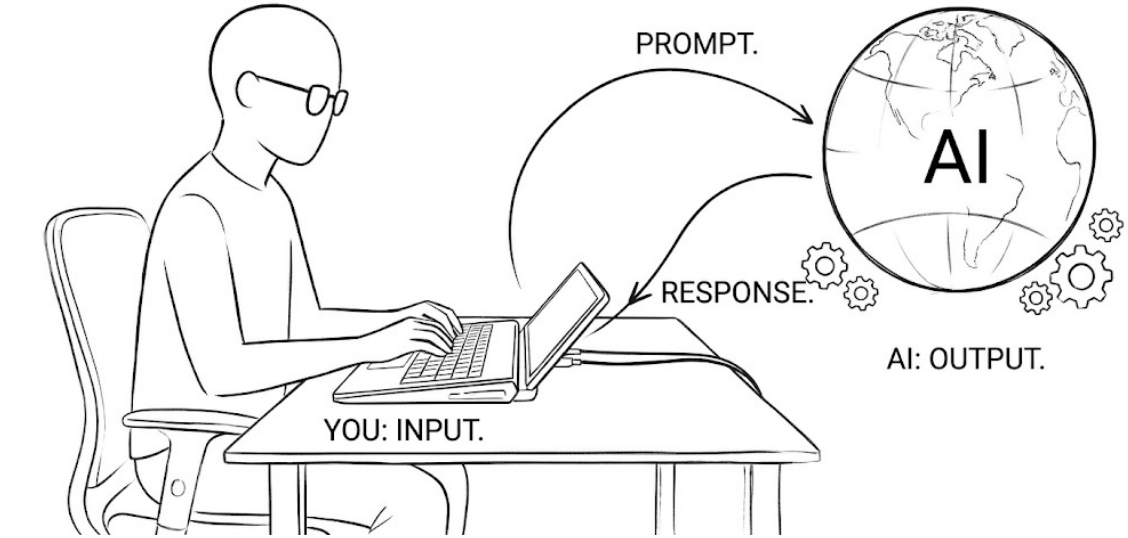
提示词工程

为什么重要?

提示工程（Prompt Engineering） 是通过精心设计、优化和精炼输入文本（Prompt），引导大语言模型（LLM）生成高质量、准确输出的技术。它本质上是人与 AI 之间的“翻译层”，将模糊的人类意图转化为精准的逻辑指令。其重要性在于，大语言模型的表现高度依赖于指令的质量。**优秀的提示工程能有效减少 AI 的“幻觉”现象，激发模型的深层推理能力，并实现低成本的任务定制。** 在无需重新训练模型的情况下，通过调整指令就能让 AI 快速胜任代码编写、创意写作或数据分析等专业角色。掌握它，意味着你能更高效地驾驭 AI，将其从简单的聊天机器人转化为真正的生产力助手。

Codex Prompt Best Practice

Claude Prompt Best Practice



提示词工程

区分提示词的好坏

写一个Java的图书管理系统。能加书、借书、还书。还要能看哪些书在。

你是一个经验丰富的Java后端开发工程师。请帮我用Java编写一个“简易图书管理系统”的核心业务逻辑。

要求如下：

1. 实体类 (Book):
   - 包含属性：`id` (String), `title` (String), `author` (String), `isBorrowed` (boolean)。
   - 提供全参构造函数、所有属性的 Getters/Setters，并重写 `toString()` 方法以便于友好打印。

2. 管理类 (Library):
   - 内部使用 `HashMap<String, Book>` 存储图书，以图书的 `id` 为键，以保证查询效率。
   - 实现以下方法：
     - `void addBook(Book book)`: 添加新书。如果传入的书籍 id 已存在，请抛出 `IllegalArgumentException`。
     - `void borrowBook(String id)`: 根据 id 借书。如果书不存在，抛出 `NoSuchElementException`；如果书已被借出，抛出 `IllegalStateException`。将状态更新为已借出。
     - `void returnBook(String id)`: 根据 id 还书。逻辑与借书类似，处理好状态变更。
     - `List<Book> listAvailableBooks()`: 过滤并返回所有 `isBorrowed == false` 的书籍列表。

3. 测试与规范:
   - 代码需符合 Java 标准编码规范（驼峰命名法）。
   - 关键逻辑请包含简短的中文注释。
   - 请提供一个 `Main` 测试类，在 `main` 方法中模拟正常的添加、借阅、归还流程，并演示触发异常的边界情况（使用 try-catch 捕获并打印异常信息）。

在Vibe-coding中使用版本控制

一个简单的示例

Main.java — ecnu-oop-java-up

源代码管理

存储库

ecnu-oop-ja...

更改

消息(⌘Enter 在"feat...

提交

更改 6

Book.class tutorials/vi... U

Book.java tutorials/vi...

Library.class tutorials/... U

Library.java tutorials/...

Main.class tutorials/vib... U

Main.java t...

图表

GITLENS

Book.java | Library.java | Main.java

tutorials > vibe_coding > good_project > Main.java > Language Support for Java(TM) by Red Hat > Main

保留 撤消

```java
import java.util.List;

public class Main {
    运行 main | 调试 main | Run | Debug
    public static void main(String[] args) {
        Library library = new Library();

        Book b1 = new Book("B001", "Java 核心技术", "Cay S. Horstmann", false);
        Book b2 = new Book("B002", "Effective Java", "Joshua Bloch", false);
        Book b3 = new Book("B003", "重构", "Martin Fowler", false);

        System.out.println("=== 添加图书 ===");
        library.addBook(b1);
        library.addBook(b2);
        library.addBook(b3);
        printAvailableBooks(library.listAvailableBooks());

        System.out.println("\n=== 正常借阅与归还流程 ===");
        library.borrowBook("B001");
        System.out.println("借阅 B001 后可借图书:");
        printAvailableBooks(library.listAvailableBooks());

        library.returnBook("B001");
        System.out.println("归还 B001 后可借图书:");
        printAvailableBooks(library.listAvailableBooks());

        System.out.println("\n=== 异常边界情况演示 ===");

        try {
            library.addBook(new Book("B001", "重复 id 的书", "Unknown", false));
        } catch (IllegalArgumentException e) {
            System.out.println("添加重复 id 异常: " + e.getMessage());
        }

        try {
```

保留 撤消 3/3

聊天 CODEX

JAVA简易图书管理系统实现指导

- listAvailableBooks：返回未借出列表（见 Library.java:38 ）
- 关键逻辑已加简短中文注释（借还书与过滤逻辑处）

3. Main 测试类

- 演示正常添加、借阅、归还流程（见 Main.java:11 ）
- 演示异常边界并用 try-catch 打印信息：
  - 重复 id 添加
  - 借不存在图书
  - 重复借阅
  - 还未借出图书
  - 还不存在图书

  （见 Main.java:27 ）

验证结果：

1. 已执行编译，结果成功（javac *.java）
2. 已执行运行，输出显示正常流程与各异常分支均按预期触发并打印异常信息。

已更改 3 个文件 +179 -3 保留 撤消

Main.java

描述要构建的内容

Agent GPT-5.3-Codex · High

本地 默认审批

feature/tutorials* 12 182 1 Connect Git Graph Indexing completed. Java: Ready 3 行 14，列 29 空格: 4 UTF-8 LF Java Prettier

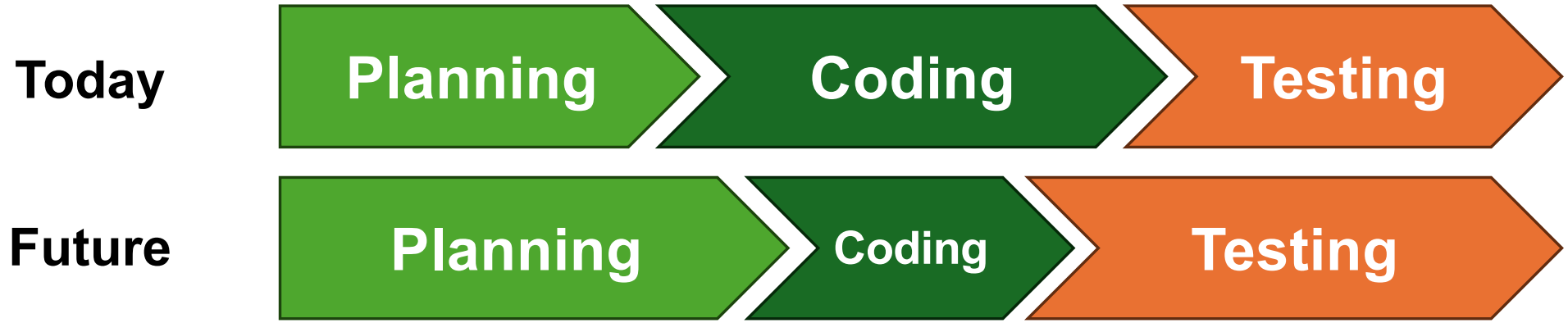
测试驱动的Vibe-coding

软件测试

什么是软件测试?

软件测试的经典定义是：**在规定的条件下对程序进行操作，以发现程序错误，衡量软件质量，并对其是否能满足设计要求进行评估的过程**[1]。

为什么vibe-coding需要软件测试? Vibe-coding依赖AI生成代码，开发者往往不逐行审查，代码质量难以保证。AI可能产生逻辑错误、安全漏洞、边界遗漏或"幻觉"代码。软件测试是最后的质量防线——通过测试验证功能正确性、发现隐藏缺陷、确保系统可靠性。越是不完全理解的代码，越需要测试来兜底。没有测试的vibe-coding，本质上就是在"盲目信任"AI[2]。



[1] Software-testing, wiki; [2] Gemini

软件测试

如何在vibe-coding过程中进行软件测试?

这里专注于软件测试的实现方式，而不是软件测试的具体方法。

测试实现方式	优点	缺点
人类先写测试，再让Agent写代码	需求意图由人类把控，Agent很难作弊	人类需要具备测试设计能力；难以覆盖人类未预见的边界情况；
Agent生成测试，人类仅审核测试用例	审核测试用例可以避免部分Agent作弊；	仍需人类介入；人类可能遗漏部分不合理的测试；
Agent独自完成开发、测试	人类参与度最低，开发速度最快	完全信任Agent

项目评分细则

Vibe-Coding

Vibe-coding项目会按照以下要求进行评分，

要求	要求	占比
参考示例项目提交完整项目文件	提供功能相关的代码	5%
	完善、**可读**的说明文档	10%
提交Vibe-coding聊天记录	与Github Copiot完成项目的所有聊天	30%
实现题目描述的项目需求	**正确**的项目源代码	30%
	正确的项目测试代码	10%
实现**有实际意义**的额外需求		15%
提出**建设性**意见		15%

Play now!